SNOWFLAKE

Site Feasibility, End to End

AstraZeneca hands-on lab

What to do, in order. One hour.

How to use this guide

This guide tells you **what to do and what you should see**. It does not explain the technology - the notebook does that, in the text above each cell, and your facilitator will talk through it as you go.

Nothing in this lab is an exercise. Every cell is written for you. You click **Run**, read the result, and move on. There is nothing to edit and nothing you can break.

If a step does not do what this guide says it will, **raise a hand**. Do not debug it. Each step can be completed even if the one before it went wrong.

What you need

Item	Detail
Account URL, username, password	Sent to you before the session
A browser	Everything happens in one tab
Anything to install	No

All data in this lab is **synthetic**. Every site, investigator, institution and study code is fictional. No AstraZeneca data is present in these accounts.

The shape of the hour

Step	You will	Mins
0	Create your workspace and open the notebook	5
1	Extract structured data from 10 survey PDFs	9
2	Normalise the item names and score the result against ground truth	7
3	Parse, chunk and search the survey text	8
4	Deploy a semantic view and query it in English	5
5	Use an agent in CoWork, then check the trace in Observability	9
6	Deploy an app and change its ranking	11
7	Wrap up	3

Step 0: Set up your workspace

Do this first. It is the only thing you build yourself.

Create the workspace

1. Left-hand navigation: **Projects**, then **Workspaces**.
2. Click **+**, then choose **From Git repository**.
3. Fill in the four fields below, then click **Create**.

Field	Enter exactly this
Repository URL	https://github.com/Alexander122776/AZ-HOL-2026-Q3.git
Workspace name	**AZ-HOL-2026-Q3**
API integration	**AZ_HOL_ASSETS_API**
Credentials	Leave empty - the repository is public

Open the notebook

4. In the workspace file list, open the **notebook** folder.
5. Open **az_feasibility_lab.ipynb**.
6. Run the first cell.

Expect this A **Connect your notebook** dialog appears the first time you run a cell. **Accept every default** and click **Create and connect**. Change nothing. It takes about a minute and happens only once.

7. Run the second cell to list the 10 survey PDFs already on your stage.

You should see: 10 PDF files. Nothing to upload - they were placed there when your account was provisioned.

Look at one survey

8. In a second browser tab: **Data**, **Databases**, **AZ_FEASIBILITY_DEMO**, **RAW**, **Stages**, **SURVEYS**.
9. Click any PDF and find the site capabilities question.

Look for: some surveys use tick boxes, others just list what the site has. Keep that in mind - it comes back in step 2.

Step 1: Extract

Run each cell in order. Read the notes above each one while it runs.

Cell	What to expect
Extract from all 10 PDFs	**1 to 2 minutes.** The slowest cell in the lab. Leave it alone.
Inspect one document	The extracted fields, each with a confidence score. Compare them against the PDF you opened in step 0.
Flatten to one row per item	Look at the **not_available** column: seven documents show 0, three show 1 or 2. That is expected, and step 2 deals with it.
Checkpoint 1	**PASS**, with 10 documents.

If the checkpoint does not pass Raise a hand. Do not rerun cells out of order.

Step 2: Normalise and score

Cell	What to expect
Show the name mismatches	Roughly a third of the items fail to match on exact text. “CT scan” against “CT Scan”, “Tumor biopsy” against “Tumour Biopsy”.
Match on embeddings	The same items now resolve to the controlled vocabulary. One SQL statement, no service to provision.
Question polarity	Read the last two columns together. Two sites answered **Yes** and it means the opposite thing in each case, purely because of how the question was worded.
Score against ground truth	**100% on both categories.** Appendix A explains why your own documents will score lower.
Checkpoint 2	**PASS**.

Step 3: Parse, chunk and search

Cell	What to expect
Parse one document	About four seconds. The document comes back as markdown.
Parse all 10	**The longest wait in the lab - a few minutes.** The cell scales the warehouse up for the parse and straight back down. Read the notes above it while you wait.
Chunk the text	Roughly 90 chunks from 20 pages, at 400 characters with 80 of overlap.
Create the search service	Returns in about ten seconds, then indexes in the background for about a minute.
Query it	The question asks for “we would need to refer patients to a third party facility”, which appears verbatim nowhere. **Every hit should be an Additional Comments section** - free text, not tick boxes.
Checkpoint 3	**PASS**, with 20 pages and roughly 90 chunks.

Step 4: Semantic view

The view is already built in your account. You will look at it as a file, deploy a copy, and query it.

Run the first two cells

You should see: 4 logical tables, 4 relationships, 22 dimensions, 10 facts, 11 metrics.

Open the model as a file

10. In the workspace file list, open **site_feasibility_test.sv.yaml**.
11. It opens in the Semantic View Editor, with a **Visual** and a **YAML** toggle.

Expect this warning: “Unable to validate. Deploy to see validation”. The file opens as a draft, so that is correct, not a fault.

Deploy it

12. Click **Deploy**.
13. Target name **SITE_FEASIBILITY_TEST**, into **AZ_FEASIBILITY_DEMO / SEMANTIC_VIEWS**.
14. Validation and the **Playground** now light up.

Before you ever click Deploy on a model of your own The deployed name comes from the **name: field inside the YAML**, not from the Target name box. Deploy a file whose name: matches an existing view and it replaces that view. Check the name: line first.

Look at four things, then ask a question

- **Logical tables** - four entities, each mapped to a physical table.
- **Relationships** - the joins, declared once.
- **Metrics** - open m_capability_coverage.
- **Custom instructions** - where the business rules live. The key one: a site with no survey is **not assessed**, not **not capable**.

15. Open the **Playground** and ask one question in plain English.
16. Read the SQL it generated.

Then run the last two cells. The final one drops SITE_FEASIBILITY_TEST, leaving your account as it started. Checkpoint should print **PASS** with 10 assessed sites.

Step 5: The agent

Do these three things in order. Do not skip straight to CoWork.

1. Look at the agent object

17. Left-hand navigation: **AI & ML**, then **Agents**.
18. Open **SITE_FEASIBILITY_AGENT**.

Look at: the three tools it can choose between, the instructions in plain language, and the access model - it runs as you, so there is no second copy of the data.

Tool	The agent uses it for
feasibility_analyst	Anything countable or comparable
survey_search	What a site actually **wrote**
code_execution	Python, for charts and composite scoring

2. Ask four questions in CoWork

19. Open **CoWork** from the left navigation (the sparkle icon).
20. Select **Site Feasibility Assistant**. It is already listed - both agents were registered with your account when it was provisioned, so there is nothing to add.
21. Ask the four questions below, in order.

After each answer, expand the **reasoning** block and click **Show SQL**.

Question 1

```
Which sites raised concerns in their survey comments, and what exactly did they write?
```

Expect: document search, not SQL.

Question 2

```
Which sites can perform Cardiac MRI, and which cannot?
```

Expect: a structured query. Check the answer separates sites that **cannot** from sites that were **never assessed**.

Question 3

```
Plot capability coverage against planned enrolment for all assessed sites
and highlight any site that is high enrolment but low coverage.
```

Expect: SQL to fetch the rows, then Python to compute and plot, then a chart. **SITE_007 Pune** and **SITE_005 Krakow** should be flagged, both at 67% coverage.

Question 4

Anything you like. Spend your remaining time here.

3. Prove the Python actually ran

The reasoning block shows what the agent **says** it did. Observability shows what it did.

22. **AI & ML**, then **Agents**, then **SITE_FEASIBILITY_AGENT**.
23. Open the **Observability** tab.
24. Sessions are listed by thread, labelled with the first question asked. Open the thread containing question 3.
25. Read the trace top to bottom and find the **Code Execution: bash** step.

A real trace from the dry run: LLM Planning 5.5s, SQL Execution 608ms, LLM Planning 6.7s, **Code Execution: bash 7.4s**, LLM Planning 1.8s, Skill: chart_instructions 1ms, LLM Planning 2.5s, Code Execution: read 7ms, LLM Planning 9.5s, Chart Generation 2.2s, LLM Response Generation 14.7s. **Total 56.1s.**

Two things to notice. Code Execution is a separate step from SQL Execution - the rows were fetched, then handed to Python. And the LLM planning steps account for roughly 40 of those 56 seconds, against 608ms of SQL.

Step 6: Deploy the app

Deploy it

26. Run the cell that lists the staged app files.
27. Run the cell that creates the app. **The first deploy takes a couple of minutes.** Listen to the explanation while it builds.

If the deploy errors Delete the **RUNTIME_NAME** and **COMPUTE_POOL** lines from the cell and run it again. Three of the four pages then work. The **Ask** page will not - raise a hand and your facilitator will demonstrate it.

Open it

28. **Projects**, then **Streamlit**, then **MY_SITE_SHORTLIST**.

Look at four pages

Page	What to look for
Portfolio	How many of each study's required items each site can provide. **Only one site in the portfolio meets every requirement of its study** - requirements differ per study.
SITE_009 New Delhi	Top of the shortlist and still short one item. The missing item is **X-Ray**, and the evidence reads **EXTRACTED** - the survey was asked and said no. Compare that with a **DERIVED_ABSENT** row, which the survey never mentioned.
Ranking weights	Switch from **Balanced** to **Requirements-heavy**. The top site changes from SITE_009 New Delhi to SITE_001 Chennai. Neither is wrong - the weights are a choice.
Document review	The signed PDF on the left, what was extracted from it on the right. This is the page that makes the other three believable.

Then ask the app one question

```
Shortlist sites for the cardiology study and tell me what the
investigators are worried about
```

Use the **Ask** page. The shortlist comes from the semantic view, the concerns from the search service, in one answer.

Checkpoint: the final cells should print **PASS** with your app deployed.

Step 7: What you built

Starting from 10 PDFs on a stage:

29. **Extracted** structured capability data straight from the page image, with a confidence score on every field.
30. **Normalised** free-text item names against a controlled vocabulary with one SQL join, respecting question polarity.
31. **Scored** the result against ground truth, so accuracy is a number rather than a hope.
32. **Modelled** it as a semantic view carrying the business rules.
33. **Asked** an agent that chose between structured query, document search and Python - and proved the Python ran.
34. **Deployed** an app on containers and changed the judgement encoded in its ranking.

That replaces a person reading PDFs into a spreadsheet, for weeks per study, with no audit trail back to the source page.

Where to take it next

- **Score a sample of your own documents against ground truth** before putting any extraction pipeline in a decision path. See Appendix A.
- **Route low-confidence extractions to human review** rather than treating all output as equal.
- **Fine-tune arctic-extract** on the layouts that fail hardest - which you will only know once you have scored them.
- **Put the pipeline behind a stream and a task**, so a new survey landing on the stage updates the portfolio without anyone opening a notebook.

Appendix A: Why your own documents will score lower

The documents in this lab were purpose-built to be unambiguous, and the pipeline scores 100% on them. That is a statement about the documents, not about the technique.

Published benchmarks on real-world form extraction typically land in the **0.7 to 0.8 F1** range. Expect to lose accuracy on:

- **Handwriting**, particularly in free-text comment boxes.
- **Scanned or photographed pages**, where skew and contrast vary page to page.
- **Inconsistent layouts** across sites, countries or survey versions.
- **Marks whose meaning depends on position** - see the table below.

Three layouts were measured while building this lab

Layout	Accuracy	Verdict
Long-form list of available items	25 of 25	Used in the lab
Checkbox images, ticked or empty	8 of 8	Used in the lab
Typed **x** in a bracket, [x] versus [ ]	**50%**	**Dropped**

The typed-x layout defaulted almost every row to available. The model could see a mark was present but could not attribute it to the right row, because the mark is a text character whose meaning depends entirely on its position relative to a bracket.

The failure was silent. Nothing errored. The pipeline returned a confident, well-formed answer that was wrong about half the time. Only scoring against ground truth exposed it.

Four things to take away

- **Measure before you trust.** An unscored extraction pipeline is a rumour.
- **Layouts where the mark is a character, not a graphic, are the hard cases.** Fine-tune on those first, and route them to human review.
- **Use the confidence scores.** They are calibrated and free. The single wrong field found while building this lab scored 0.32, against 0.65 or higher on every correct one.
- **Keep join keys out of extraction.** A misread identifier corrupts every downstream join silently. Take it from the filename, the folder or the source system.